

STATEMENT VS PREPAREDSTATEMENT

	Statement	PreparedStatement
Seguridad	✗ Vulnerable SQL Injection	✓ Parámetros seguros
Rendimiento	Compila cada vez	Precompilado, reutilizable
Legibilidad	Concatena strings	Parámetros con ?
Tipos de dato	Todo como String	setInt, setString...
Cuándo usar	Nunca con datos externos	Siempre con datos del usuario

MÉTODOS SET*() DE PREPAREDSTATEMENT

setString(i, val) parámetro de tipo String	setInt(i, val) parámetro de tipo int
setDouble(i, val) parámetro de tipo double	setFloat(i, val) parámetro de tipo float
setBoolean(i, val) parámetro de tipo boolean	setNull(i, tipo) inserta NULL en la BD
setDate(i, date) parámetro de tipo Date SQL	clearParameters() limpia todos los parámetros

Los índices de los parámetros **?** empiezan en 1, no en 0. El primer **?** es el índice 1, el segundo el 2, etc.

✗ ERRORES COMUNES – BLOQUES 2 Y 3

- B2** No cerrar la conexión → *resource leak*, agotamiento del pool de conexiones
- B2** Usar `executeUpdate()` con un SELECT → lanza `SQLException`
- B2** URL mal formada o BD inexistente → `SQLException: Unknown database`
- B3** Olvidar asignar algún parámetro **?** → `SQLException` al ejecutar
- B3** Usar índice 0 en `setString(0, ...)` → los índices empiezan en 1
- B3** Tipo incorrecto: `setString` para una columna `INT` → error de conversión
- B3** Crear el `PreparedStatement` pero olvidar llamar a `executeUpdate()` / `executeQuery()` → la operación no se ejecuta

inyección SQL – EL PROBLEMA Y LA SOLUCIÓN

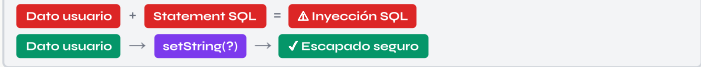
Código vulnerable con Statement:

```
// Si el usuario introduce: ' OR '1'='1'
String nombre = "' OR '1'='1'";
String sql = "SELECT * FROM usuarios WHERE nombre = '"
    + nombre + "'";

// Resultado → devuelve TODOS los registros
// Consulta real: WHERE nombre = '' OR '1'='1'
```

Solución con PreparedStatement:

```
String sql = "SELECT * FROM usuarios WHERE nombre = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, nombre); // el ? se escapa automáticamente
ResultSet rs = ps.executeQuery();
// Aunque nombre = "' OR '1'='1'", lo trata como texto literal
```



REUTILIZAR PREPAREDSTATEMENT – PATRÓN EFICIENTE

La consulta se precompila **una sola vez**. Cambiar los parámetros y volver a ejecutar es mucho más rápido que crear un nuevo Statement cada vez.

```
String sql = "INSERT INTO usuarios (nombre) VALUES (?)";
PreparedStatement ps = conn.prepareStatement(sql);
// Precompilado una vez → reutilizar N veces

String[] nombres = {"Ana", "Luis", "Marta"};
for (String n : nombres) {
    ps.setString(1, n);
    ps.executeUpdate(); // ejecuta con cada nombre
}
ps.close();
```

SISTEMA DE LOGIN SEGURO CON PREPAREDSTATEMENT

```
String sql = "SELECT * FROM usuarios WHERE nombre=? AND pass=?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, usuario);
ps.setString(2, password);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
    System.out.println("Login correcto");
} else {
    System.out.println("Credenciales incorrectas");
}
```

DAM · PROGRAMACIÓN · JDBC

Conexión JDBC y PreparedStatement

BLOQUES 2 Y 3 · CHULETARIO · TRÍPTICO A4

FLUJO JDBC COMPLETO

```
DriverManager.getConnection(url, user, pass)
↓
Connection conn
↓
Statement      PreparedStatement
conn.createStatement()  conn.prepareStatement(sql)
↓                      ↓
executeQuery()      set*(i, valor) ...
↓                      ↓
executeUpdate()  executeQuery() / executeUpdate()
↓                      ↓
ResultSet      ResultSet
rs.next() / rs.getX()  rs.next() / rs.getX()
```

`close(rs) → close(stmt/ps) → close(conn)`

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq.** — **B2** Conexión, SQLException, Statement, executeQuery/Update
- Interior cent.** — **B2** Ejemplo completo + try-with-resources · **B3** PreparedStatement SELECT
- Interior der.** — **B3** PreparedStatement INSERT, UPDATE, DELETE completos
- Solapa** — Inyección SQL: problema y solución, reutilizar PS, login seguro
- Contraportada** — Tabla comparativa, set*() completo, errores comunes

REGLA DE USO

Statement Solo para SQL fijo sin datos del usuario. Nunca concatenes variables externas.	PreparedStatement Siempre que haya datos variables o del usuario. Más seguro y eficiente.
--	---

CARA A → Exterior · CARA B → Interior (teoría y referencia)
Imprimir: A4 horizontal · doble cara · voltear por lado largo

🔑 DRIVERMANAGER Y CONNECTION B2

- **DriverManager** — gestiona los drivers y crea conexiones
- **Connection** — representa la sesión activa con la BD
- Siempre cerrar con `conn.close()` al terminar

ESTABLECER CONEXIÓN BÁSICA

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

try {
    Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/prueba",
        "root", "1234");

    System.out.println("Conexión correcta");
    conn.close();

} catch (SQLException e) {
    System.out.println(e.getMessage());
}
```

⚠️ SQLEXCEPTION B2

Excepción comprobada principal de JDBC. Siempre hay que capturarla o declararla con `throws`.

<code>e.getMessage()</code> descripción legible del error	<code>e.getErrorCode()</code> código de error del SGBD
<code>e.getSQLState()</code> código de estado estándar SQL	<code>e.printStackTrace()</code> traza completa del error

🗃️ STATEMENT B2

Usar solo para SQL estático sin datos del usuario. Vulnerable a inyección SQL si se concatenan variables.

CREAR STATEMENT Y EJECUTAR SELECT

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
    "SELECT * FROM usuarios");

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " - " + rs.getString("nombre"));
}
```

EJECUTAR INSERT, UPDATE, DELETE

```
int filas;

filas = stmt.executeUpdate(
    "INSERT INTO usuarios VALUES (4, 'Laura')");

filas = stmt.executeUpdate(
    "UPDATE usuarios SET nombre='Carlos' WHERE id=1");

filas = stmt.executeUpdate(
    "DELETE FROM usuarios WHERE id=2");

System.out.println("Filas afectadas: " + filas);
```

Método	Para	Devuelve
<code>executeQuery(sql)</code>	SELECT	<code>ResultSet</code> con los datos
<code>executeUpdate(sql)</code>	INSERT / UPDATE / DELETE / CREATE	<code>int</code> filas afectadas

✅ TRY-WITH-RESOURCES – CIERRE AUTOMÁTICO B2

Java 7+. Cierra automáticamente `Connection`, `Statement` y `ResultSet` al salir del bloque, aunque haya excepción.

```
try (
    Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/prueba",
        "root", "1234");
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(
        "SELECT * FROM usuarios")
) {
    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
// ✓ rs, stmt y conn se cierran automáticamente
```

🗃️ PREPAREDSTATEMENT – INTRODUCCIÓN B3

- Consultas **precompiladas** → más rápidas en ejecuciones repetidas
- Parámetros con `?` → nunca concatenar strings de usuario
- Tipos estrictos → `setString()`, `setInt()`...
- Índices empiezan en 1, no en 0

PREPAREDSTATEMENT SELECT CON PARÁMETRO

```
String sql = "SELECT * FROM usuarios WHERE id = ?";

try (
    Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/prueba",
        "root", "1234");
    PreparedStatement ps = conn.prepareStatement(sql)
) {
    ps.setInt(1, 1); // sustituye el primer ?
    ResultSet rs = ps.executeQuery();

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

SELECT CON DOS PARÁMETROS

```
String sql =
    "SELECT * FROM usuarios WHERE ciudad=? AND edad>?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, "Madrid"); // primer ?
ps.setInt(2, 18);           // segundo ?
ResultSet rs = ps.executeQuery();
```

🗃️ PREPAREDSTATEMENT – INSERT B3

```
String sql =
    "INSERT INTO usuarios (id, nombre) VALUES (?, ?)";
PreparedStatement ps = conn.prepareStatement(sql);

ps.setInt(1, 5); // primer ? = id
ps.setString(2, "Mario"); // segundo ? = nombre
int filas = ps.executeUpdate();
System.out.println("Insertadas: " + filas);
```

🗃️ PREPAREDSTATEMENT – UPDATE B3

```
String sql =
    "UPDATE usuarios SET nombre = ? WHERE id = ?";
PreparedStatement ps = conn.prepareStatement(sql);

ps.setString(1, "Lucia"); // nuevo nombre
ps.setInt(2, 1);          // id a modificar
ps.executeUpdate();
```

🗃️ PREPAREDSTATEMENT – DELETE B3

```
String sql = "DELETE FROM usuarios WHERE id = ?";
PreparedStatement ps = conn.prepareStatement(sql);

ps.setInt(1, 3); // id a eliminar
ps.executeUpdate();
```

📊 TABLA RESUMEN – MÉTODOS DE EJECUCIÓN

Operación	Método	Devuelve
SELECT	<code>ps.executeQuery()</code>	<code>ResultSet</code>
INSERT	<code>ps.executeUpdate()</code>	<code>int</code> filas
UPDATE	<code>ps.executeUpdate()</code>	<code>int</code> filas
DELETE	<code>ps.executeUpdate()</code>	<code>int</code> filas
CREATE/DROP	<code>stmt.executeUpdate()</code>	<code>int</code> (o)

✅ BUENAS PRÁCTICAS – BLOQUES 2 Y 3

Usar try-with-resources cierra automático y seguro	PreparedStatement siempre con datos variables o del usuario
No hardcodear credenciales usar fichero de config en prod.	Tipos correctos set*(i/v) setInt para INT, setString para VARCHAR
Cerrar en orden inverso rs → stmt/ps → conn	Reutilizar PreparedStatement precompilar una vez, ejecutar N veces

Regla de oro: usa siempre `PreparedStatement` con `?` y `set*()` cuando el SQL lleva datos del usuario. Nunca concatenes strings en consultas SQL. Cierra con try-with-resources.